

ADVANCED OPERATING SYSTEMS

Lab 1 — Kernel development and debugging

Vittorio Zaccaria

Politecnico di Milano

Introduction to the environment

This repo organization

A few words on the organisation of this repo. Where we get the kernel from, what is a initramfs (hint: initramfs is used as the first root filesystem that your machine has access to) and what you can produce with it. Our machine will not need another disk; this initial file system is everything we need to play with the kernel.

“Initramfs is setup using busybox: The Swiss Army Knife of Embedded Linux”

Building the development environment

Build the supported amd64 full image once from the repository root:

```
T=amd64 E=full make build-container
```

Then choose either frontend. Both edit the same bind-mounted repository and use the same kernel, module build, initramfs, and QEMU.

Host terminal and LazyVim

```
T=amd64 E=full make dev-up
T=amd64 E=full make dev-build
T=amd64 E=full make dev-vi F=modules/lab-1-intro-hello-module/module.c
```

The full image provides LazyVim and clangd. Exit LazyVim with `:qa`.

Browser editor

```
make vscode-build-container
make vscode-up
```

Open `http://127.0.0.1:8080`, enter the password `aos-linux-labs`, and open `modules/lab-1-intro-hello-module/module.c` from the Explorer. VS Code edits the host repository at `/repo`; clangd diagnostics and navigation run inside the container. If port 8080 is already in use, start it with `make VSCODE_PORT=8081 vscode-up` and open that port instead.

Running and rebuilding

With the host-terminal frontend, rebuild and start the generated kernel and initramfs with:

```
T=amd64 E=full make dev-build
T=amd64 E=full make dev-run
```

With the browser frontend, use **Terminal → Run Build Task** for `AOS: Build modules and initramfs`. Then use **Terminal → Run Task** and select `AOS: Run QEMU (1 vCPU)`. The task terminal is the QEMU guest console. Whenever you change a module, run the build task again before booting.

Modules

Modules and module lifecycle

- Modules are a **flexible way of handling** the operating system **image** at runtime.
- Instead of rebooting with a different operating system image, modules allow easy extension of the operating systems' capabilities as required.
- It is the most convenient way with which we are going to play with the kernel.

See lab-1-intro-hello-module

Module parameters

- Parameters can be declared and initialised in code

```
static int num = 5;
/* S_IRUG0: everyone can read the sysfs entry */
module_param(num, int, S_IRUG0);
```

- And defined at module instantiation time

```
insmod yourmodule.ko num=10
```

Who executes the module?

Loading a module does not create a process. Who calls its initialization and cleanup functions?

The hello module logs the current task in both functions:

```
#include <linux/sched.h>

/* Inside hello_init(): */
pr_info("hello_init: comm=%s pid=%d\n",
        current->comm, task_pid_nr(current));

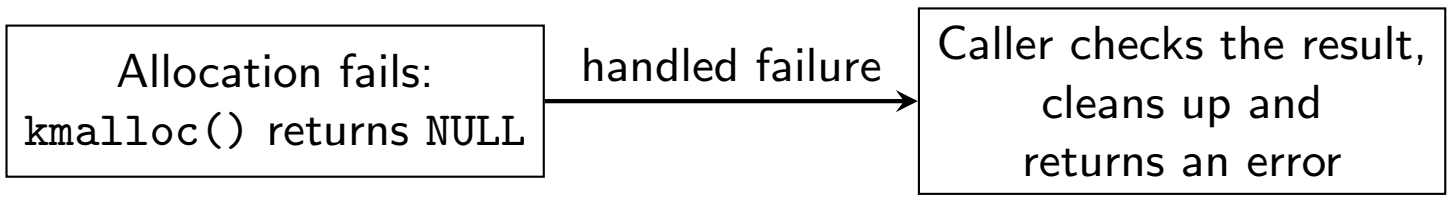
/* Inside hello_cleanup(): */
pr_info("hello_cleanup: comm=%s pid=%d\n",
        current->comm, task_pid_nr(current));
```

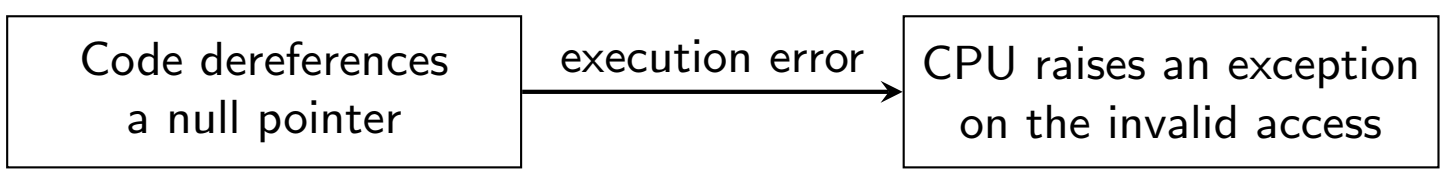
`current` identifies the task executing this kernel code; `comm` is its name.

Module debugging

Kernel failures > an error return is not an oops

An expected failure is handled through normal control flow. A programming defect can instead make kernel execution invalid.



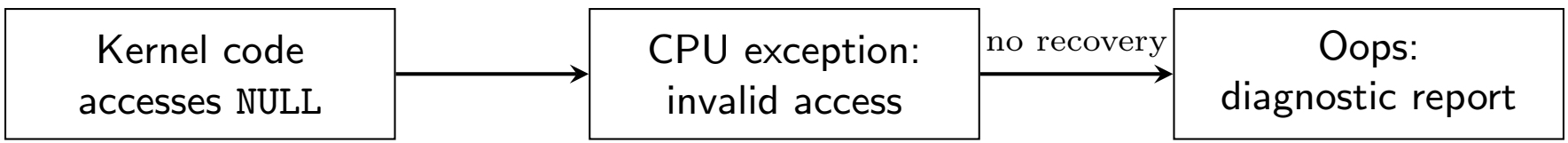


A failed allocation can lead to the second case if the caller dereferences the null result instead of checking it.

Not every invalid access is detected immediately: a write to mapped memory may silently corrupt another object.

Kernel failures > what is an oops?

An oops is Linux's handling of a serious kernel execution error when it cannot recover the failing execution path. It produces a diagnostic report and abandons that path.

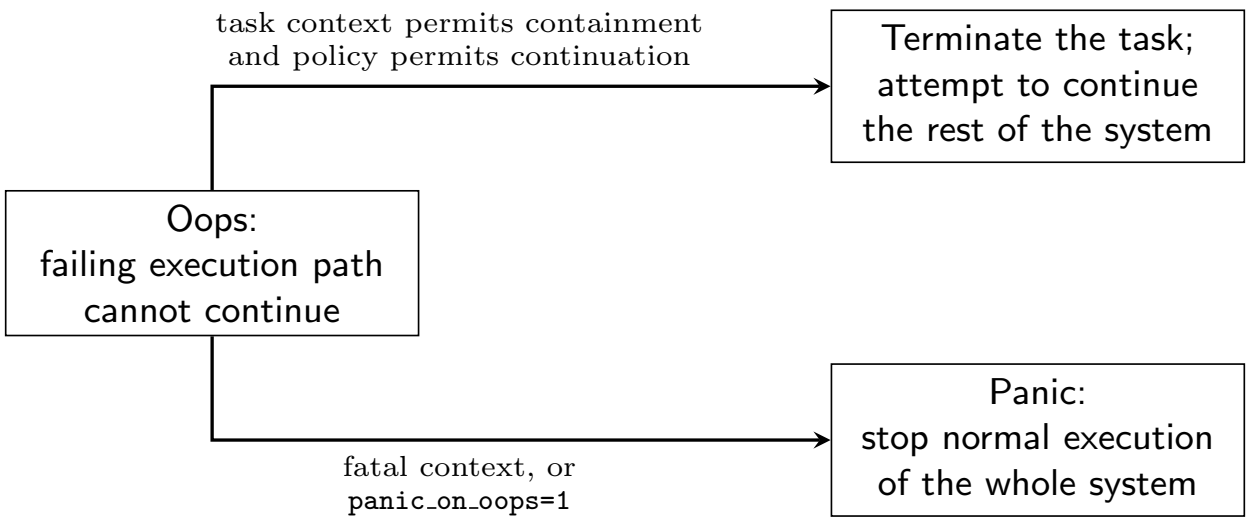


- The **CPU exception** is the hardware event; the **oops** is the kernel's error handling and report.
- The report identifies the failing instruction, relevant registers and call trace.
- An oops does **not** by itself mean that the whole system has stopped or rebooted.

Some kernel exceptions have recovery paths. For example, supported user-memory access helpers can handle invalid user addresses; those faults need not cause an oops.

Kernel failures > when does an oops lead to panic?

Panic means that the kernel stops normal system-wide execution. Whether an oops leads to panic depends on context and policy.



If the error occurs while servicing an interrupt, the kernel cannot simply terminate a calling user task to contain it.

Continuing after an oops does not guarantee consistent state. After a panic, configuration determines crash dumping, halting or rebooting. Code can also call `panic()` directly, without a preceding oops.

Kernel failures > reading the oops report

Real QEMU capture: Linux `5.16.0-rc1`, temporary module writing through a null pointer.

REPORT EXCERPT	INTERPRETATION

`BUG: kernel NULL pointer dereference,`
`address: 0000000000000000`
`#PF: supervisor write access in kernel mode`
`Oops: 0002 [#1] PREEMPT SMP NOPTI`
	Error and address
	A kernel-mode write to address zero.
	`#PF` means page fault.

`CPU: 0 PID: 77 Comm: insmod` `Tainted: G O 5.16.0-rc1 #1`	**Execution environment** CPU, task, kernel version and taint.
`RIP: 0010:oops_demo_init+0x5/0x1000` `[oops_demo]`	**Failing instruction** `RIP`: function + byte offset, then the containing module.

`RSP: 0018:ffffc900001d7cb0 EFLAGS: 00000246`
`RAX: 0000000000000000 RBX: 0000000000000000`
`RCX: 0000000000000000`
	Register state
	Stack pointer, flags and data registers.
	Decode the instruction before
	assigning a meaning to each value.

`Call Trace:`
`<TASK>`
`do_one_initcall+0x44/0x200`
`[...]`
`load_module+0x2381/0x26b0`
`[...]`
	Call path
	Module initialization was reached
	through the module loader.

`Modules linked in: oops_demo(O+)`	**Module state** `O`: out-of-tree; `+`: being loaded.

Excerpt: timestamps removed; lines selected and wrapped. `[...]` omits frames. Full report.

Poorman's printk

Printing from kernel to the log buffer is done today with the `pr_*()` family of functions:

- `pr_emerg(), pr_alert(), pr_crit(), pr_err(), pr_warn(), pr_notice(), pr_info(), pr_cont()`,

For example: `pr_info("Booting CPU %d\n", cpu);`

For pointers:

- `%p`: Display the hashed value of pointer by default.
- `%px`: Always display the address of a pointer (use carefully on non-sensitive addresses).
- `%pK`: Display hashed pointer value, zeros or the pointer address depending on kptr_restrict sysctl value

See for example

- How to get printk format specifiers right — The Linux Kernel documentation
- bootlin.com/doc/training/debugging/debugging-slides.pdf

The hard truth about kernel debugging

- Usually no magic formula, requires **creative detective work**
- This lab is not about this creative part (this requires time) but the tools that are available to you
- In general many ways in which you can arrive to the same result

Introduction to GDB

Introduction

Why use live GDB

- Understanding the code flow
- Dumping data structures and assembly
- Debugging hangs

Why not use live gdb

- Issue is not reproducible
- Don’t know what to look for

Including debug info in your kernel

You must use `CONFIG_DEBUG_INFO` in your `.config` file otherwise..

```
(gdb) c
Continuing.
^C
Thread 1 received signal SIGINT, Interrupt.
Oxffffffff81e8ccbf in default_idle ()
(gdb) bt
#0 Oxffffffff81e8ccbf in default_idle () <----- No line info!
#1 Oxffffffff81e8cf8c in default_idle_call()
#2 Oxffffffff810da469 in do_idle ()
```

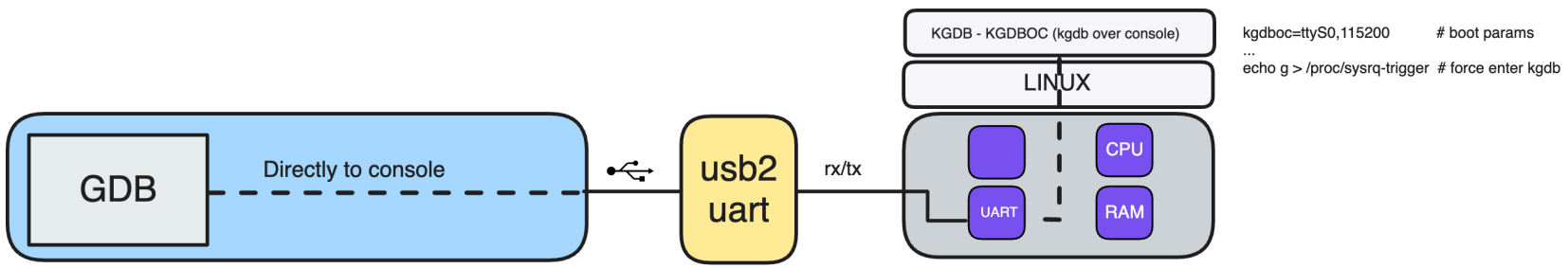
Disabling KSLR

You must boot with `nokaslr` your kernel otherwise you'll get random and unintelligible stack frame info

```
(gdb) c
Continuing.
^C
Thread 1 received signal SIGINT, Interrupt.
0xffffffff81e8ccbf in ??
(gdb) bt
#0 0xffffffff81e8ccbf in ?? <----- No symbol info!
#1 0xffffffff81e8cf8c in ??
#2 0xffffffff810da469 in ??
```

External hardware > KGDB

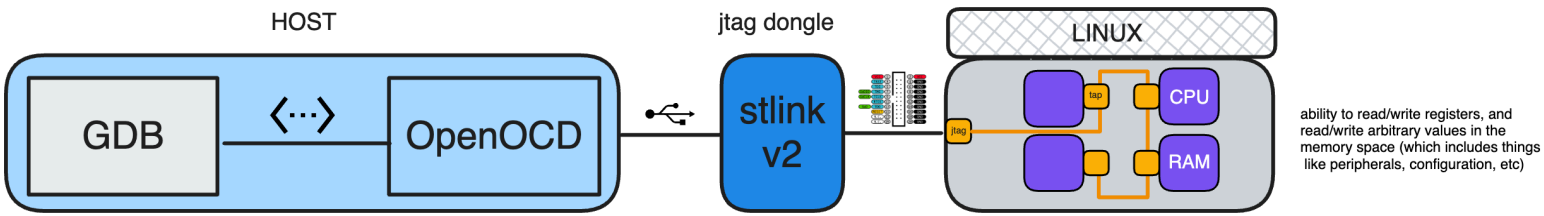
The execution of the kernel can be fully controlled by gdb from another machine, connected through a serial line (enabled by `CONFIG_KGDB_SERIAL_CONSOLE`).



"GDB connected over a serial port to a live Linux machine."

It can do almost everything, including inserting breakpoints in interrupt handlers and provides GDB python scripts to ease debugging.

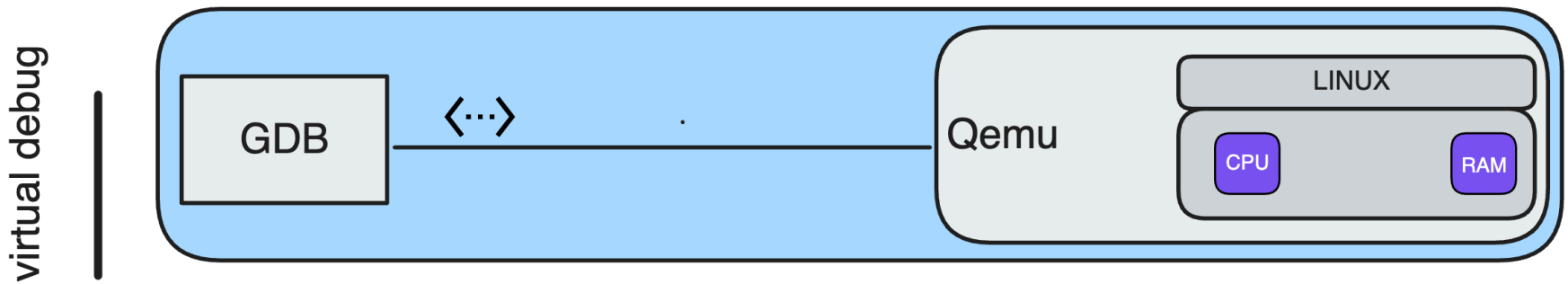
External hardware > JTAG



“GDB connected through JTAG to a live Linux machine; KGDB is not required.”

JTAG is allows to debug a system with almost no functioning software. You use JTAG on system bring up if you are not sure that you even reach a point where kgdb could work.

Internal emulation > QEMU



“GDB connected directly to QEMU; no intermediary is required.”

With QEMU, we could emulate access to our linux system as it was over a jtag, even if kgdb is not enabled. GDB commands are the same though.

Mini tutorial on GDB for the kernel

Preliminaries > GDB commands > most used

```
break <sym> # inserts a break point
info break  # shows breakpoints
delete <num># deletes <num> breakpoint
continue    # continues from a breakpoint to the next
step        # enters the subroutine and breaks
next        # does not enter subroutines and breaks
bt          # prints stack backtrace
```

Preliminaries > GDB commands > Linux specific scripts

```
lx-lsmod # list modules
lx-ps    # list processes
apropos lx

p $lx_current()
p *(struct task_struct *) <address of process from lx-ps>
p $container_of(listheadp, "struct <type x>", "name of the list head in <type X")
p $lx_task_by_pid(<PID>)
set $t = $lx_task_by_pid(<PID>)
p $t -> __state # print state
```

Anatomy of a syscall > attach to the running guest

Already at the amd64 QEMU shell; the hello module is built but **not loaded**.

This walkthrough uses terminal GDB. With the browser frontend, use the graphical kernel and module debugging procedure near the end of this handout instead.

QEMU monitor: press `Ctrl-]`, then `c`.

```
gdbserver tcp:127.0.0.1:1234
```

Use the same keys to return to the guest console.

Second host terminal: `make dev-sh`**, then**

```
cd /sources/linux
make scripts_gdb
gdb -q \
  -iex "add-auto-load-safe-path $PWD" \
  vmlinux
```

GDB

```
set pagination off
target remote 127.0.0.1:1234
p sys_call_table[175]
hbreak __x64_sys_init_module
hbreak do_init_module
continue
```

On x86-64, **175** is `init_module`.

Now run in the QEMU guest

```
insmod /modules/lab-1-intro-hello-module.ko
num=7
```

GDB stops before the syscall executes.

Anatomy of a syscall > caller and arguments

At `__x64_sys_init_module`, `regs` points to the **user registers saved at syscall entry**, not GDB's current CPU registers.

GDB at the first breakpoint

```
p $lx_current()->pid
p $lx_current()->comm
p regs->orig_ax
p/x regs->di
p/d regs->si
x/s regs->dx
x/4bx regs->di
```

The syscall arguments are:

```
init_module(image, length, parameters)
```

`di`: image pointer; `si`: length in bytes; `dx`: parameter-string pointer.

Observed in the remote run

```
pid:       80
comm:      "insmod..."
orig_ax:   175
di:        0x7fb6044fd010
si:        228288
string:    " \"num=7\""
ELF bytes: 0x7f 0x45 0x4c 0x46
```

PID, addresses and image length vary. The string includes BusyBox's added spaces and quotes.

The four bytes identify an **ELF image**. The task remains `insmod` while executing kernel code.

Anatomy of a syscall > initialization and return

GDB

```
continue
p mod->name
bt
finish
finish
finish
p/d $rax
detach
quit
```

At `do_init_module`, `mod->name` is `"module"`.

On this build, the three `finish` commands return from `do_init_module`, `load_module`, then `__do_sys_init_module`.

Observed call path — selected frames:

```
do_init_module
load_module
__do_sys_init_module
do_syscall_x64
do_syscall_64
entry_SYSCALL_64
```

Each `finish` returned **0** `$rax` was **0** at the syscall return to `do_syscall_x64`.

Guest after detach: status `0`, module `Live`.

```
echo $?
cat /proc/modules
rmmod module
```

Timer callback > stop before it runs

After the syscall exercise, use a **fresh guest and a fresh GDB session**. The timer module must already be built into the initramfs.

Reuse the monitor and GDB startup commands from the syscall slide. In GDB:

```
set pagination off
target remote 127.0.0.1:1234
hbreak do_init_module
continue
```

Guest

```
insmod /modules/lab-1-list-manip.ko
```

GDB stops before module initialization. The timer has not been armed yet.

GDB at `do_init_module`

```
p mod->name
p $lx_current()->comm
lx-symbols /repo/modules/lab-1-list-manip
p/x &my_timer_handler
hbreak *my_timer_handler
continue
```

Observed: module `"module"`, task `"insmod"`.

The `*` selects the function's **entry address**, before its body executes. Install this breakpoint before continuing; the timer's three-second delay is not setup time.

Timer callback > who is executing now?

At the first callback entry

```
bt
p $lx_current()->pid
p $lx_current()->comm
x/i $pc
p timer
p/x timer->function
```

Observed PID **0**, name **`swapper/0`**. `timer->function` matched the breakpoint address.

`current` identifies the interrupted task. It does not mean that task requested the callback; a different run may interrupt another task.

Observed stack — selected frames:

```
my_timer_handler
__run_hrtimer
__hrtimer_run_queues
hrtimer_interrupt
__sysvec_apic_timer_interrupt
sysvec_apic_timer_interrupt
```

Unlike the syscall path, this callback is entered from a timer interrupt. GDB stopped unwinding at the interrupt-stack boundary in this run.

End the demonstration here. Exit QEMU with `Ctrl-]`, then `x`; then `quit` GDB. Do not `continue`, `detach` or `rmmod` in this demonstration: the callback still contains the intentional list-traversal fault.

Optional > debugging the list fault

The separate list-debugging exercise starts from a fresh VM. It is not needed to demonstrate timer execution context.



After removing the tail, `cur` stays unchanged and `cur->next` is null. The next loop iteration evaluates `cur->next->next`.

The callback allocates with `GFP_ATOMIC` and skips traversal if allocation fails; initialization uses `GFP_KERNEL` and frees any partial list on failure. Cleanup cancels the timer before freeing the list. The traversal fault remains intentional: do not treat this as a correct list implementation.

Optional > post-fault analysis with GDB

After the panic, GDB attaches to QEMU at the panic path, not at the faulting instruction. The original `struct pt_regs` is still available through the `regs` parameter of the `page_fault_oops` frame:

```
target remote 127.0.0.1:1234
lx-symbols /repo/modules/lab-1-list-manip
bt
frame N                    # select page_fault_oops; N varies
set $fault = regs
info line *$fault->ip
x/8i $fault->ip-16
p/x $fault->di
p *(struct list_node *)$fault->bx
p/x &head                  # address of the global variable
p/x head                   # pointer to the first node
p *head                    # first node
```

In the source, `head` is the global `struct list_node *head` ; `lx-symbols` gives GDB its name and type. In the verified run, saved `RDI` was zero. On x86, `CR2` contains the virtual address that caused the latest page fault: the `next` field is at offset 8, so `0x8(%rdi)` with `RDI=0` explains `CR2=8` . `CR2` is not the page-fault error code. The

disassembly showed that saved `RBX` held `cur`; inspecting it recovered `cur->next == NULL`. Confirm the compiler's register assignment from the disassembly before interpreting a saved register.

Optional > graphical kernel and module debug in the browser

With the browser frontend running, first execute the `AOS: Build modules and initramfs` task. Then:

1. run the `AOS: Debug QEMU (4 vCPUs)` task;
2. open **Run and Debug**, select `AOS: Attach to QEMU kernel`, and press `F5`;
3. inspect the stopped `start_kernel` frame, then press `F5` again to continue booting.

To stop in the hello module initialization, enter these commands in the Debug Console before loading the module:

```
aos-module-symbols /repo/modules/lab-1-intro-hello-module
break hello_init
```

Continue the kernel, then run `insmod /modules/lab-1-intro-hello-module.ko` in the QEMU guest. The helper resolves the module's runtime section addresses before `hello_init` executes. GDB, QEMU, and the source tree all remain inside the browser container; no separate `gdbfrontend` process or host GDB port is required.

Linkography

- Kernel debugging with QEMU
- Provoking crashes with LKDTM
- Linux kernel debugging
- Debugging the kernel using GDB
- bootlin.com/doc/training/debugging/debugging-slides.pdf
- Serial, KDB and KGDB debugging